Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

«Запросы на выборку и модификацию данных. Представления. Работа с индексами»

Обучающийся Сафонова Арина Дмитриевна
Факультет прикладной информатики
Группа K3241
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель работы: овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) **с использованием подзапросов**.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

- Составить список всех 2-местных номеров отелей, с ценой менее 200 т.р., упорядочив данные в порядке уменьшения стоимости.

```
SELECT
    h.name_hotel AS "Отель",
    r.number_room AS "Номер",
    tr.count_places AS "Количество мест",
    p.price_per_day AS "Цена за сутки"
FROM hotel.room r
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
JOIN hotel.type_room tr
    ON r.id_type_room = tr.id_type_room
JOIN hotel.price p
    ON tr.id_type_room = p.id_room_type
WHERE tr.count_places = 2
  AND p.price_per_day < 200000
ORDER BY p.price_per_day DESC;
```

Query | Query History

```sql
SELECT
    h.name_hotel AS "Отель",
    r.number_room AS "Номер",
    tr.count_places AS "Количество мест",
    p.price_per_day AS "Цена за сутки"
FROM hotel.room r
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
JOIN hotel.type_room tr
    ON r.id_type_room = tr.id_type_room
JOIN hotel.price p
    ON tr.id_type_room = p.id_room_type
WHERE tr.count_places = 2
  AND p.price_per_day < 200000
ORDER BY p.price_per_day DESC;
```

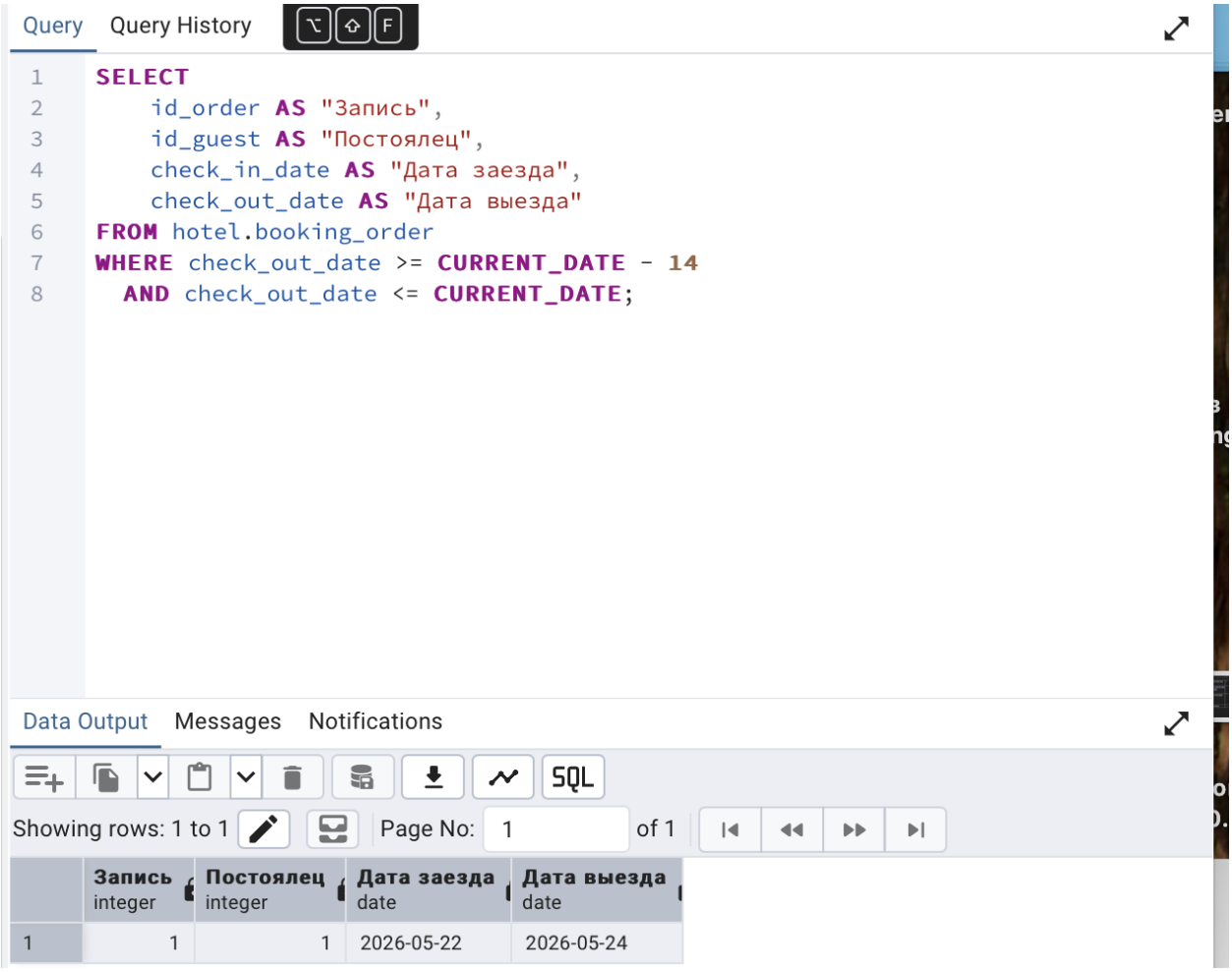
Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	Отель character varying	Номер integer	Количество мест integer	Цена за сутки numeric (10,2)
1	Grand Hotel	102	2	5000.00
2	Nevsky Palace	202	2	5000.00

- Выбрать все записи регистрации постояльцев, которые выехали из отелей в течение двух последних недель.

```sql
SELECT
  id_order AS "Запись",
  id_guest AS "Постоялец",
  check_in_date AS "Дата заезда",
  check_out_date AS "Дата выезда"
FROM hotel.booking_order
WHERE check_out_date >= CURRENT_DATE - 14
 AND check_out_date <= CURRENT_DATE;
```



- Чему равен общий суточный доход каждого отеля за последний месяц?

```
SELECT
    h.name_hotel AS "Отель",
    b.check_out_date AS "Дата",
    SUM(b.amount) AS "Суточный доход"
FROM hotel.booking_order b
JOIN hotel.room r
    ON b.id_room = r.id_room
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
WHERE b.check_out_date >= CURRENT_DATE - 30
  AND b.check_out_date <= CURRENT_DATE
GROUP BY h.name_hotel, b.check_out_date;
```

Query | Query History

```sql
SELECT
    h.name_hotel AS "Отель",
    b.check_out_date AS "Дата",
    SUM(b.amount) AS "Суточный доход"
FROM hotel.booking_order b
JOIN hotel.room r
    ON b.id_room = r.id_room
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
WHERE b.check_out_date >= CURRENT_DATE - 30
  AND b.check_out_date <= CURRENT_DATE
GROUP BY h.name_hotel, b.check_out_date;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	Отель character varying	Дата date	Суточный доход numeric
1	Grand Hotel	2026-05-...	10000
2	Nevsky Palace	2026-05-...	8000

- Составить список свободных номеров заданного отеля на текущий день.

```sql
SELECT
    h.name_hotel AS "Отель",
    r.number_room AS "Номер",
    r.floor AS "Этаж",
    r.status_room AS "Статус"
FROM hotel.room r
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
WHERE h.name_hotel = 'Grand Hotel'
  AND r.status_room = 'free';
```

Query | Query History

```sql
SELECT
    h.name_hotel AS "Отель",
    r.number_room AS "Номер",
    r.floor AS "Этаж",
    r.status_room AS "Статус"
FROM hotel.room r
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
WHERE h.name_hotel = 'Grand Hotel'
  AND r.status_room = 'free';
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	Отель character varying	Номер integer	Этаж integer	Статус character varying
1	Grand Hotel	101	1	free

- Найти общие потери от незанятых номеров за текущий день по всей сети.

```sql
SELECT
   SUM(p.price_per_day) AS "Общие потери"
FROM hotel.room r
JOIN hotel.type_room tr
   ON r.id_type_room = tr.id_type_room
JOIN hotel.price p
   ON tr.id_type_room = p.id_room_type
WHERE r.status_room = 'free';
```

Query | Query History

```sql
SELECT
    SUM(p.price_per_day) AS "Общие потери"
FROM hotel.room r
JOIN hotel.type_room tr
    ON r.id_type_room = tr.id_type_room
JOIN hotel.price p
    ON tr.id_type_room = p.id_room_type
WHERE r.status_room = 'free';
```

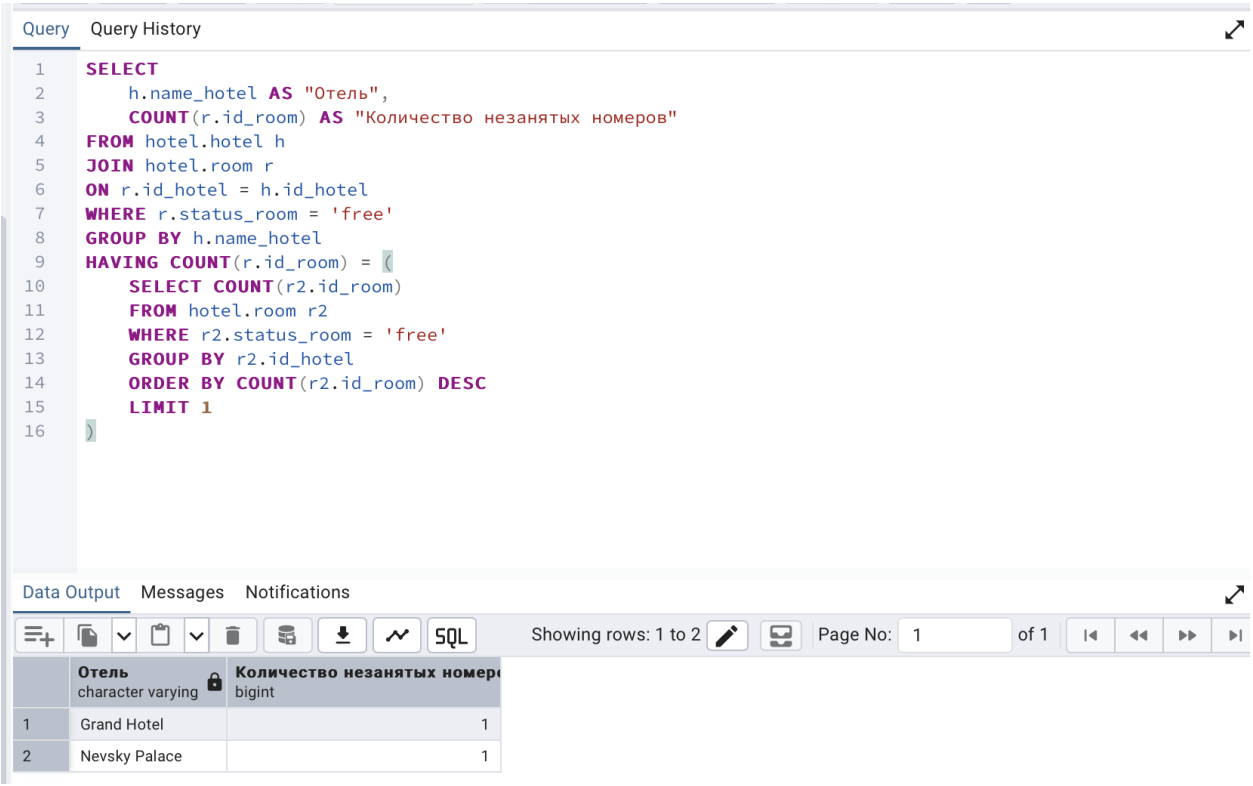
Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	Общие потери numeric
1	6000.00

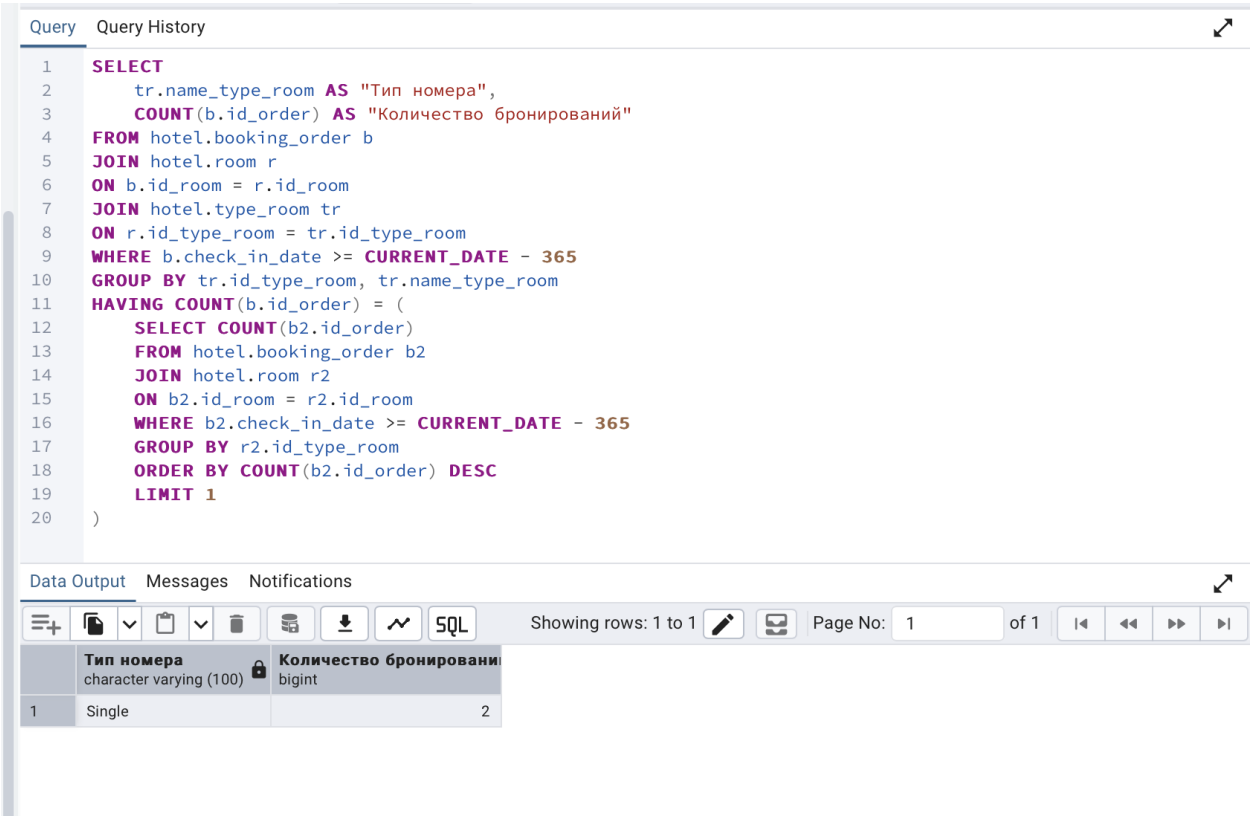
- Определить, в каком отеле имеется наибольшее количество незанятых номеров на текущие сутки.

```sql
SELECT
    h.name_hotel AS "Отель",
    COUNT(r.id_room) AS "Количество незанятых номеров"
FROM hotel.hotel h
JOIN hotel.room r
ON r.id_hotel = h.id_hotel
WHERE r.status_room = 'free'
GROUP BY h.name_hotel
HAVING COUNT(r.id_room) = (
    SELECT COUNT(r2.id_room)
    FROM hotel.room r2
    WHERE r2.status_room = 'free'
    GROUP BY r2.id_hotel
    ORDER BY COUNT(r2.id_room) DESC
    LIMIT 1
)
```



- Определить самый популярный тип номеров за последний год.

```
SELECT
    tr.name_type_room AS "Тип номера",
    COUNT(b.id_order) AS "Количество бронирований"
FROM hotel.booking_order b
JOIN hotel.room r
ON b.id_room = r.id_room
JOIN hotel.type_room tr
ON r.id_type_room = tr.id_type_room
WHERE b.check_in_date >= CURRENT_DATE - 365
GROUP BY tr.id_type_room, tr.name_type_room
HAVING COUNT(b.id_order) = (
    SELECT COUNT(b2.id_order)
    FROM hotel.booking_order b2
    JOIN hotel.room r2
    ON b2.id_room = r2.id_room
    WHERE b2.check_in_date >= CURRENT_DATE - 365
    GROUP BY r2.id_type_room
    ORDER BY COUNT(b2.id_order) DESC
    LIMIT 1
)
```



Задание 3. Создайте представления:

- Для турагентов (поиск свободных номеров в отелях).

```
CREATE VIEW  hotel.view_free_rooms_for_agents AS

SELECT

    h.name_hotel AS "Отель",

    h.address AS "Адрес",

    r.number_room AS "Номер",

    r.floor AS "Этаж",

    tr.name_type_room AS "Тип номера",

    tr.count_places AS "Количество мест",

    r.status_room AS "Статус",

    p.price_per_day AS "Цена за сутки"

FROM hotel.room r

JOIN hotel.hotel h

    ON r.id_hotel = h.id_hotel

JOIN hotel.type_room tr

    ON r.id_type_room = tr.id_type_room

JOIN hotel.price p
```

```
    ON tr.id_type_room = p.id_room_type

WHERE r.status_room = 'free';
```

Query | Query History

```
CREATE VIEW  hotel.view_free_rooms_for_agents AS
SELECT
    h.name_hotel AS "Отель",
    h.address AS "Адрес",
    r.number_room AS "Номер",
    r.floor AS "Этаж",
    tr.name_type_room AS "Тип номера",
    tr.count_places AS "Количество мест",
    r.status_room AS "Статус",
    p.price_per_day AS "Цена за сутки"
FROM hotel.room r
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
JOIN hotel.type_room tr
    ON r.id_type_room = tr.id_type_room
JOIN hotel.price p
    ON tr.id_type_room = p.id_room_type
WHERE r.status_room = 'free';
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 57 msec.
```

Query | Query History

```
SELECT *
FROM hotel.view_free_rooms_for_agents;
```

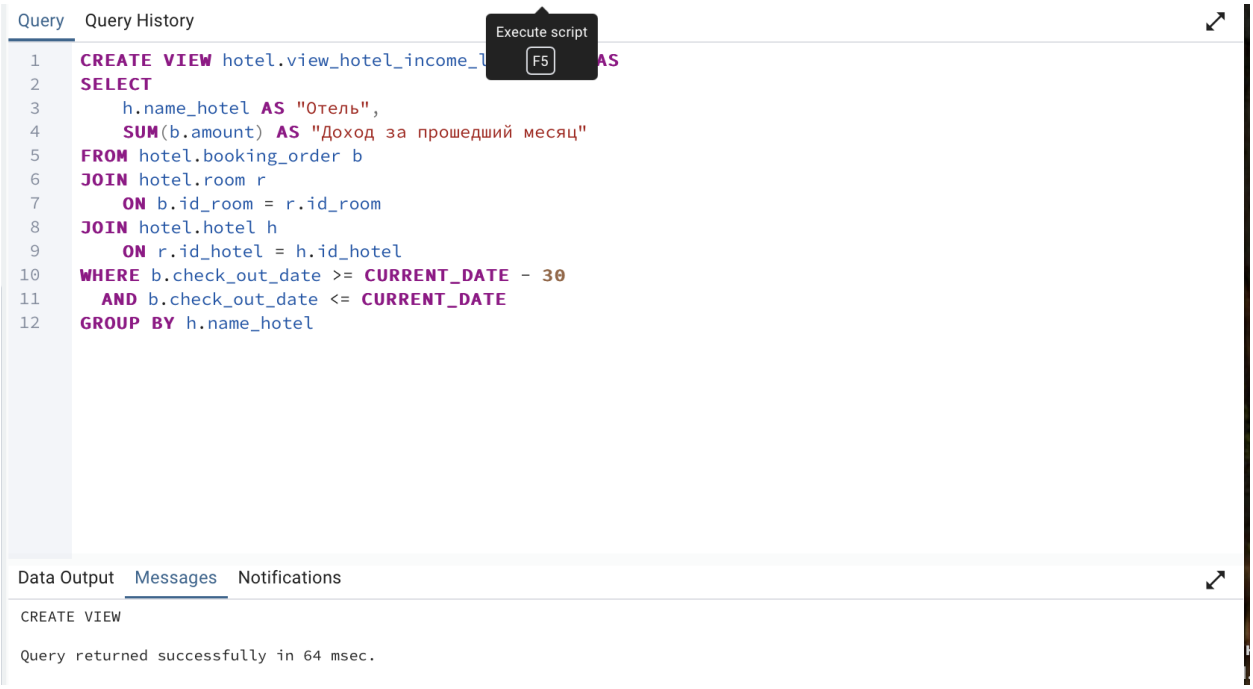
Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	Отель character varying	Адрес character varying	Номер integer	Этаж integer	Тип номера character varying (100)	Количество мест integer	Статус character varying	Цена за сутки numeric (10,2)
1	Nevsky Palace	Nevsky prospect ...	101	1	Single	1	free	3000.00
2	Grand Hotel	Tverskaya street ...	101	1	Single	1	free	3000.00

- Для владельца компании (информация о доходах каждого отеля в сети за прошедший месяц).

```
CREATE VIEW hotel.view_hotel_income_last_month AS
SELECT
    h.name_hotel AS "Отель",
    SUM(b.amount) AS "Доход за прошедший месяц"
FROM hotel.booking_order b
JOIN hotel.room r
    ON b.id_room = r.id_room
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
WHERE b.check_out_date >= CURRENT_DATE - 30
  AND b.check_out_date <= CURRENT_DATE
GROUP BY h.name_hotel
```



Query Query History

```
1  SELECT *
2  FROM hotel.view_hotel_income_last_month
```

Data Output Messages Notifications

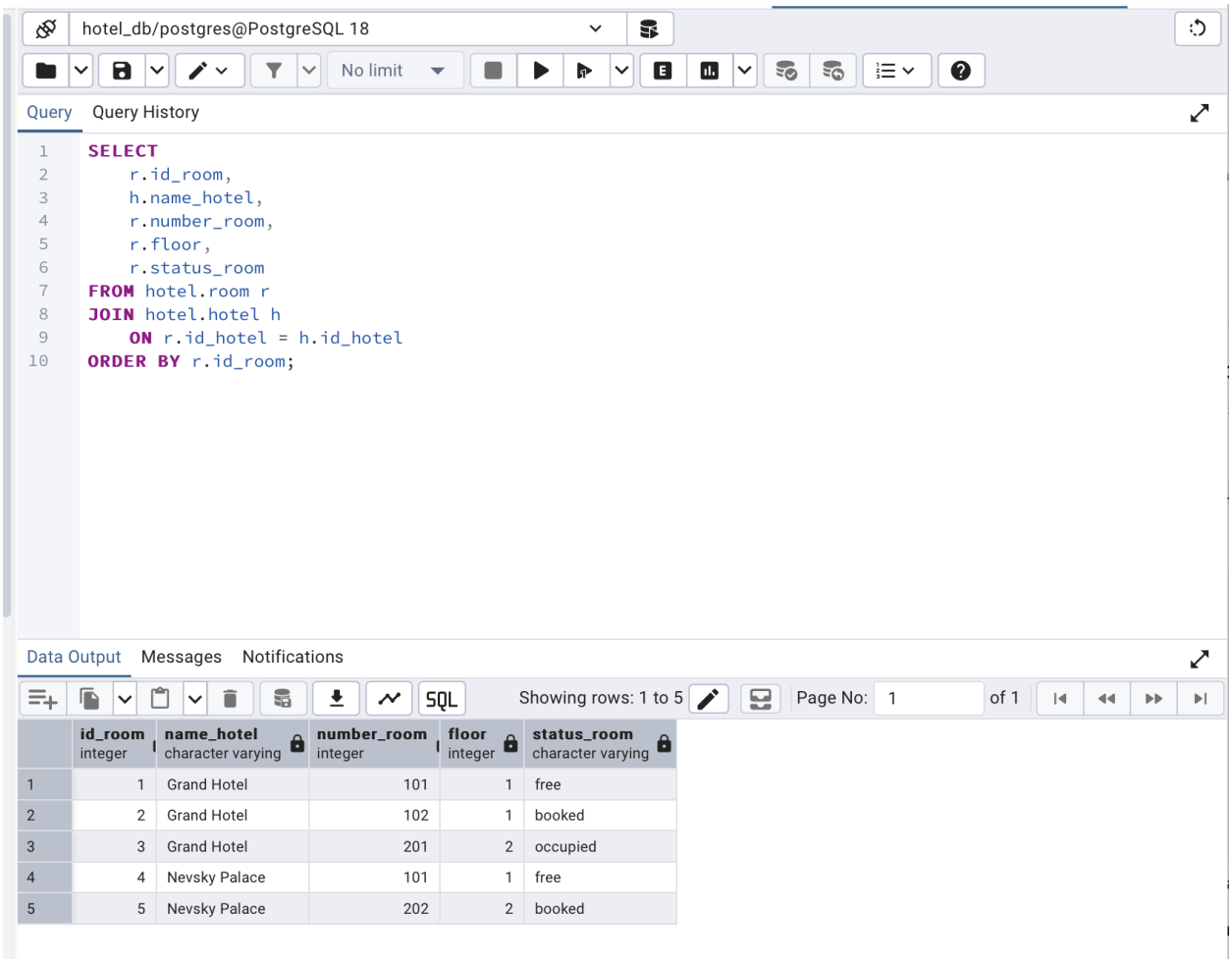
Showing rows: 1 to 2 Page No: 1 of 1

	Отель character varying	Доход за прошедший меся numeric
1	Grand Hotel	10000
2	Nevsky Palace	8000

Добавить новый номер 301 в отель Grand Hotel. Идентификатор отеля определить с помощью подзапроса.

Состояние таблицы до INSERT

```
SELECT
    r.id_room,
    h.name_hotel,
    r.number_room,
    r.floor,
    r.status_room
FROM hotel.room r
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
ORDER BY r.id_room;
```



INSERT с подзапросом

```
INSERT INTO hotel.room(
id_room,
id_hotel,
id_type_room,
floor,
number_room,
status_room
)
VALUES(
6,
(SELECT id_hotel
FROM hotel.hotel
 WHERE name_hotel = 'Grand Hotel'
),
1,
3,
301,
'free'
)
```

Query Query History

```
INSERT INTO hotel.room(
id_room,
id_hotel,
id_type_room,
floor,
number_room,
status_room
)
VALUES(
6,
(SELECT id_hotel
FROM hotel.hotel
 WHERE name_hotel = 'Grand Hotel'
),
1,
3,
301,
'free'
)
```

Data Output Messages Notifications

```
INSERT 0 1

Query returned successfully in 127 msec.
```

В результате появился номер 301

Query Query History

```
SELECT
    r.id_room,
    h.name_hotel,
    r.number_room,
    r.floor,
    r.status_room
FROM hotel.room r
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
ORDER BY r.id_room;
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	id_room integer	name_hotel character varying	number_room integer	floor integer	status_room character varying
1	1	Grand Hotel	101	1	free
2	2	Grand Hotel	102	1	booked
3	3	Grand Hotel	201	2	occupied
4	4	Nevsky Palace	101	1	free
5	5	Nevsky Palace	202	2	booked
6	6	Grand Hotel	301	3	free

Изменить статус номера 301 отеля Grand Hotel на booked. Идентификатор отеля определить с помощью подзапрос

состояние до UPDATE

Query | Query History

```sql
SELECT
    r.id_room,
    h.name_hotel,
    r.number_room,
    r.floor,
    r.status_room
FROM hotel.room r
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
WHERE r.number_room = 301
  AND h.name_hotel = 'Grand Hotel';
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	id_room integer	name_hotel character varying	number_room integer	floor integer	status_room character varying
1	6	Grand Hotel	301	3	free

update запрос для смены статуса с free на booked

```sql
UPDATE hotel.room
SET status_room = 'booked'
WHERE number_room = 301
  AND id_hotel = (
      SELECT id_hotel
      FROM hotel.hotel
      WHERE name_hotel = 'Grand Hotel'
  )
```

Query Query History

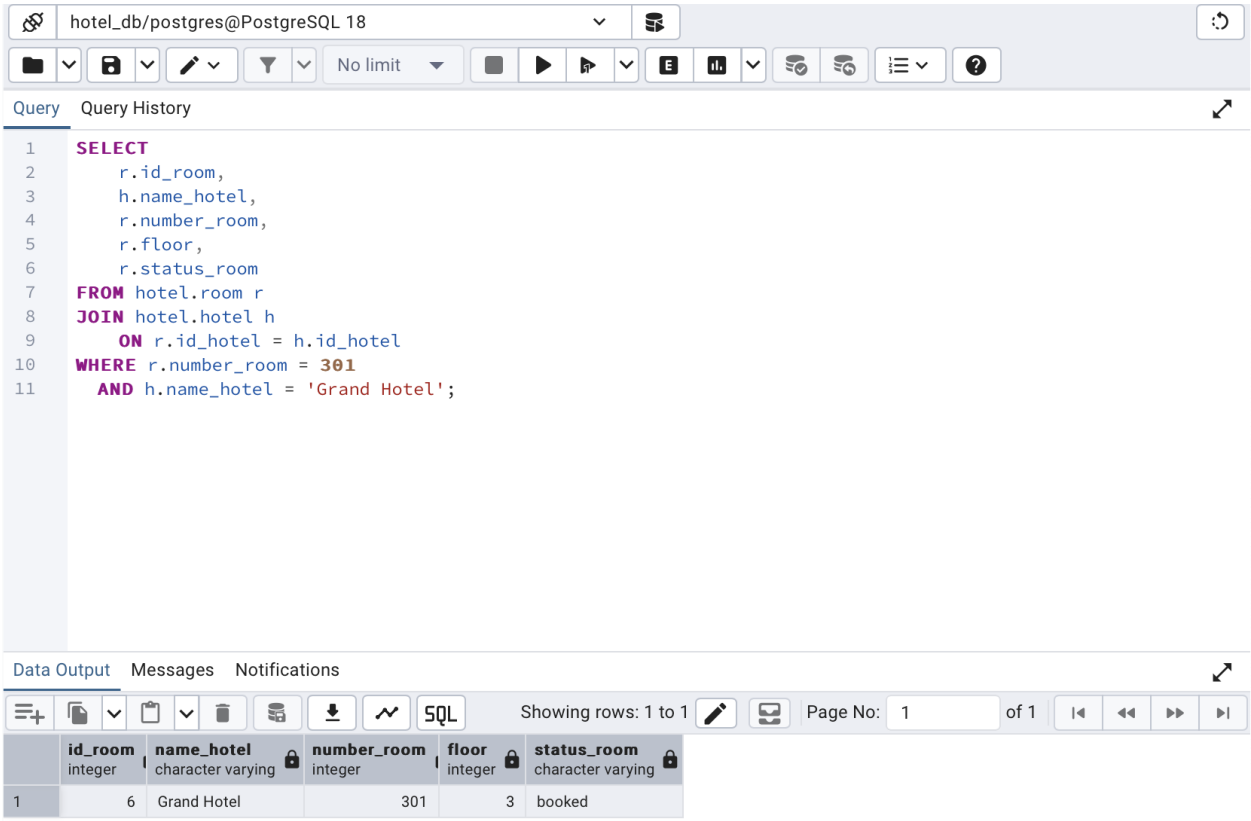
```
1  UPDATE hotel.room
2  SET status_room = 'booked'
3  WHERE number_room = 301
4    AND id_hotel = (
5        SELECT id_hotel
6        FROM hotel.hotel
7        WHERE name_hotel = 'Grand Hotel'
8    )
```

Data Output Messages Notifications

```
UPDATE 1

Query returned successfully in 99 msec.
```

После UPDATE



Удалить номер 301 из отеля Grand Hotel. Идентификатор отеля определить с помощью подзапроса.

Доказательство существования 301 номера

Query Query History

```sql
SELECT
    r.id_room,
    h.name_hotel,
    r.number_room,
    r.floor,
    r.status_room
FROM hotel.room r
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
WHERE r.number_room = 301
  AND h.name_hotel = 'Grand Hotel';
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	id_room integer	name_hotel character varying	number_room integer	floor integer	status_room character varying
1	6	Grand Hotel	301	3	booked

```sql
DELETE FROM hotel.room
WHERE number_room = 301
  AND id_hotel = (
      SELECT id_hotel
      FROM hotel.hotel
      WHERE name_hotel = 'Grand Hotel'
  )
```

Query Query History

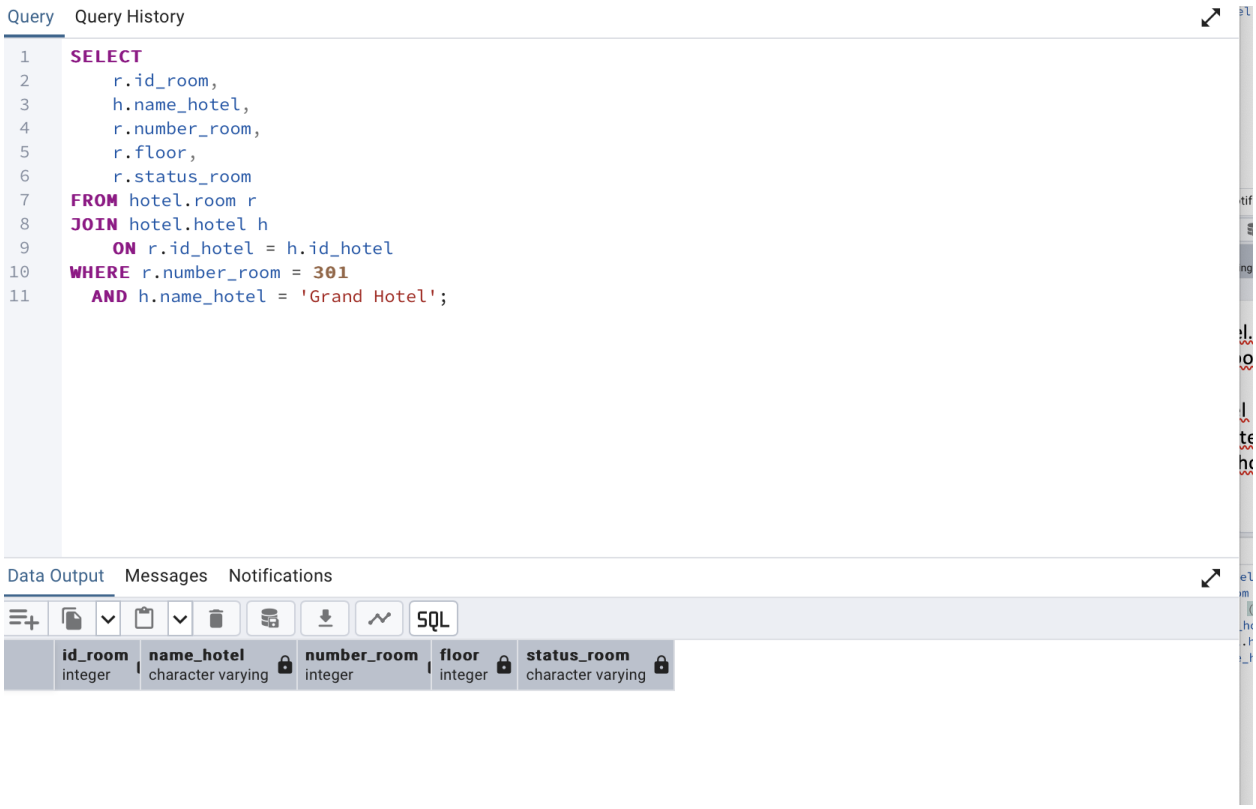
```sql
DELETE FROM hotel.room
WHERE number_room = 301
  AND id_hotel = (
      SELECT id_hotel
      FROM hotel.hotel
      WHERE name_hotel = 'Grand Hotel'
  )
```

Data Output Messages Notifications

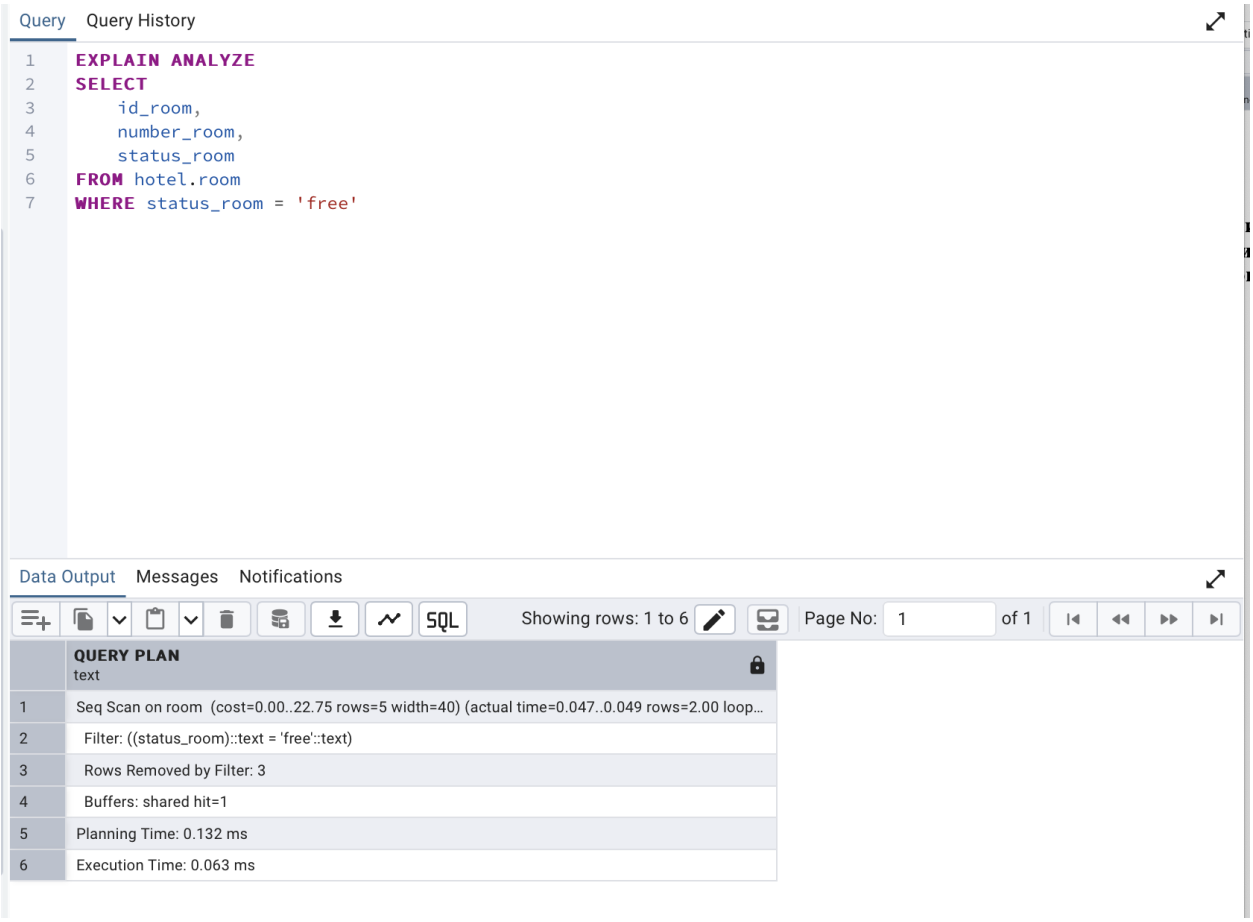
```
DELETE 1

Query returned successfully in 104 msec.
```

После DELETE номер удален

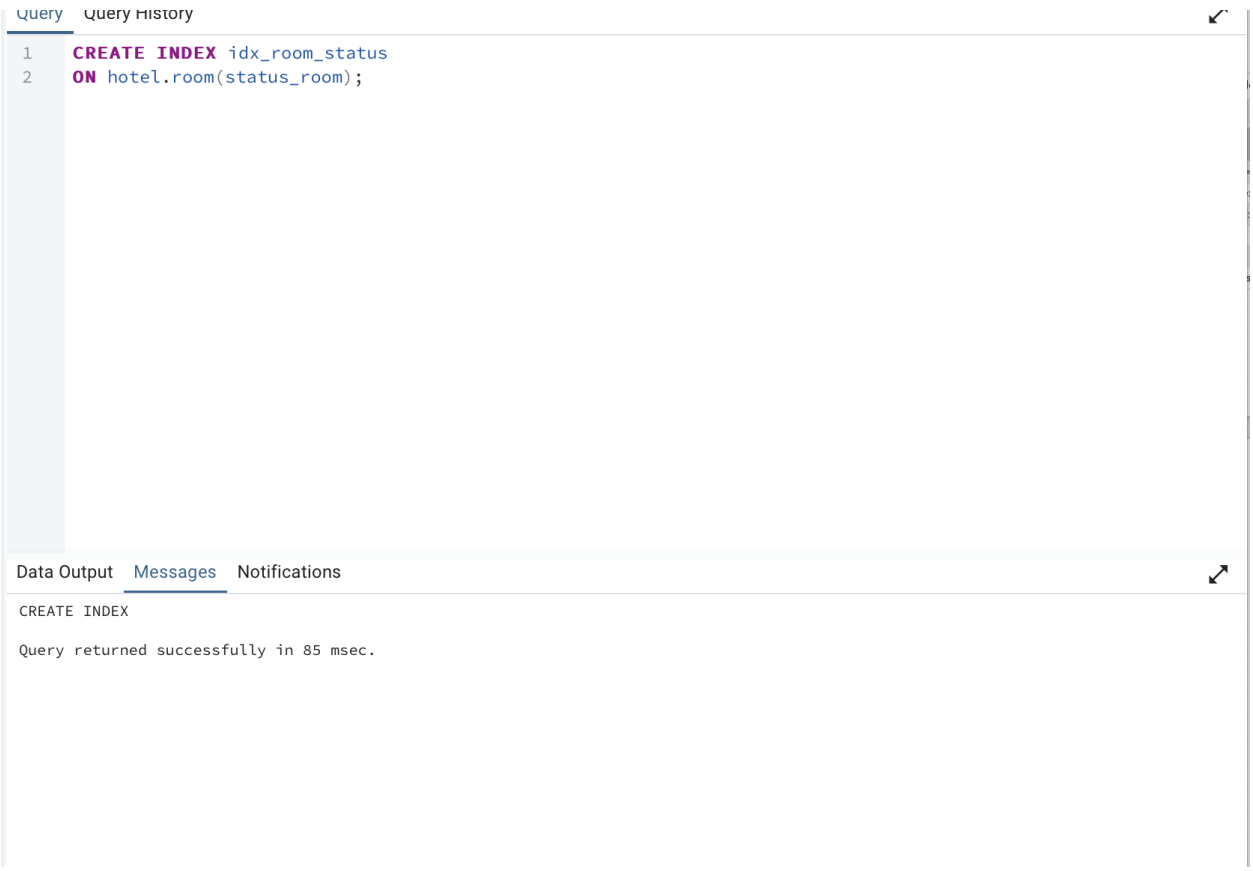


Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.



Создание индекса

CREATE INDEX idx_room_status
ON hotel.room(status_room);



После создания простого индекса по полю status_room время выполнения запроса уменьшилось с 0.063 ms до 0.039 ms. Однако в плане выполнения остался Seq Scan, так как таблица содержит небольшое количество строк, и PostgreSQL считает последовательное сканирование более выгодным, чем использование индекса.

Анализ плана выполнения запроса без составного индекса

Для анализа выполнения запроса была использована команда `EXPLAIN ANALYZE`, так как по методике лабораторной работы необходимо получить план запроса и сравнить время выполнения запроса без индекса и с индексом

```
EXPLAIN ANALYZE
SELECT
   id_room,
   number_room,
   floor,
   status_room
FROM hotel.room
WHERE id_hotel = 1
  AND status_room = 'free'
```

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT
3      id_room,
4      number_room,
5      floor,
6      status_room
7  FROM hotel.room
8  WHERE id_hotel = 1
9    AND status_room = 'free'
```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on room (cost=0.00..1.07 rows=1 width=44) (actual time=0.027..0.028 rows=1.00 loop...
2	Filter: ((id_hotel = 1) AND ((status_room)::text = 'free'::text))
3	Rows Removed by Filter: 4
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=6 dirtied=2
7	Planning Time: 0.858 ms
8	Execution Time: 0.057 ms

CREATE INDEX idx_room_hotel_status
ON hotel.room(id_hotel, status_room);

Query Query History

```
1  CREATE INDEX idx_room_hotel_status
2  ON hotel.room(id_hotel, status_room);
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 60 msec.
```

Проверяем время после сотавного индекса

```
EXPLAIN ANALYZE
SELECT
   id_room,
   number_room,
   floor,
   status_room
FROM hotel.room
WHERE id_hotel = 1
 AND status_room = 'free';
```

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT
3      id_room,
4      number_room,
5      floor,
6      status_room
7  FROM hotel.room
8  WHERE id_hotel = 1
9    AND status_room = 'free';
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on room (cost=0.00..1.07 rows=1 width=44) (actual time=0.027..0.028 rows=1.00 loop...
2	Filter: ((id_hotel = 1) AND ((status_room)::text = 'free'::text))
3	Rows Removed by Filter: 4
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=19 read=1
7	Planning Time: 1.368 ms
8	Execution Time: 0.057 ms

После создания составного индекса заметного ускорения не произошло:
время выполнения осталось тем же — **0.057 ms**.

Это связано с тем, что таблица room содержит очень небольшое количество строк, поэтому PostgreSQL считает более выгодным выполнить **последовательное сканирование (**Seq Scan**)**, чем использовать индекс.

Таким образом, составной индекс был успешно создан, однако на данной небольшой учебной таблице его влияние на производительность практически не проявилось.